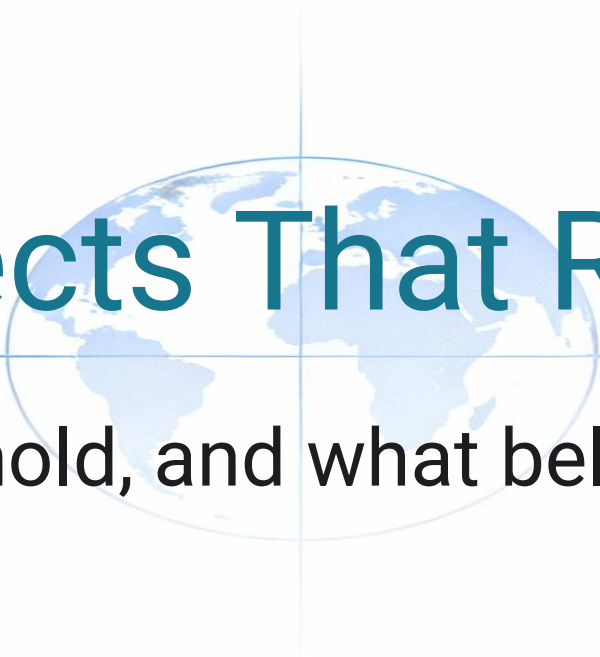




# Objects That Refuse

Rules that hold, and what belongs to whom

Evis Plaku



## Your Account **accepts anything**

- Every one of these compiles and runs today
- Once an object is born wrong it stays wrong for its whole life
- And it gets passed around, added to lists, and totalled

```
// all of these work right now
new Account("AC-1001", null, -5000.00);
new Account("", "", 0.00);
new Account("AC-1002", "Ada", -1.00);

// an account owned by nobody,
// holding negative money
```

## An **invariant** holds for the whole life of the object

### Born broken, and it stays broken

`new Account(null, -5000)`    `deposit(100)`    `withdraw(50)`    `getBalance()`

balance is negative from the first instant, and every later answer is wrong

### Born valid, and every method keeps it valid

`new Account("Ada", 500)`    `deposit(100)`    `withdraw(50)`    `getBalance()`

balance  $\geq 0$  holds at every single one of these points

That statement is the invariant. It must be true when the object is born, and no method may leave it false.



## Bad data in, no object out

- throw stops the constructor. The object never exists
- So there is no half-built account to find later
- Put the bad value in the message. Invalid input tells you nothing

```
public Account(String accountNumber,  
               String ownerName,  
               double openingBalance) {  
  
    this.accountNumber = requireText(  
        accountNumber, "Account number");  
    this.ownerName = requireText(  
        ownerName, "Owner name");  
  
    if (openingBalance < 0) {  
        throw new IllegalArgumentException(  
            "Opening balance cannot be "  
            + "negative: " + openingBalance);  
    }  
    this.balance = openingBalance;  
}
```



## A guard that does not stop anything **is not a guard**

- It complains, and then assigns the bad value anyway
- The account is created, the balance is negative, nothing stopped
- This is worse than no check, because it looks like validation

```
// what almost everyone writes first
if (openingBalance < 0) {
    System.out.println(
        "Balance cannot be negative");
}
this.balance = openingBalance;

// a comment with extra steps
```



The same check was about to appear **six times**

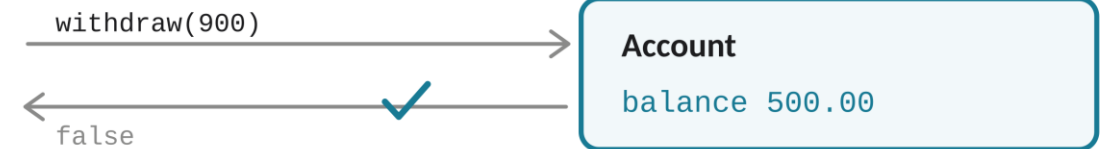
- So write it once and call it from everywhere
- The argument from week 1, now applied to your own code
- `isBlank` catches forty spaces. `isEmpty` does not

```
private String requireText(String value,  
                           String label) {  
    if (value == null || value.isBlank())  
    {  
        throw new  
        IllegalArgumentException(  
            label + " is required");  
    }  
    return value;  
}
```



One of these is normal. The other is a bug

- A customer asking for too much is a normal thing that happens
- A negative amount is a bug. No interface offers one
- Those two deserve completely different answers



Normal. The caller decides what to tell the customer.



A bug in the caller. Stop now, loudly, with the value in the message.



The gate holds. There is no broken account to find later.



## deposit no longer returns anything

- Last week it returned a boolean that could only ever be true
- A return value meaning "you passed me rubbish" is better thrown
- withdraw keeps its boolean, because refusing is a real outcome

```
public void deposit(double amount) {  
    if (amount <= 0) {  
        throw new  
        IllegalArgumentException(  
            "Deposit must be positive: "  
            + amount);  
    }  
    balance += amount;  
}
```



## A setter that earned its place

- People change their name and the bank records it. A real operation
- setBalance would hand back the power that private took away
- Ask of every public method: is this something the thing does?

```
public void setOwnerName(String name) {  
    this.ownerName =  
        requireText(name, "Owner name");  
}  
  
// still no setBalance,  
// and there never will be  
  
// the balance moves through deposit  
// and withdraw, or it does not move
```



## A getter and a setter for every field **protects nothing**

- Count what it actually stops. Anyone can still read and write anything
- It is the same open data with four extra lines per field
- Write setters when the operation is real, not by reflex

### A getter and a setter for every field

|                    |                    |
|--------------------|--------------------|
| getAccountNumber() | setAccountNumber() |
| getOwnerName()     | setOwnerName()     |
| getBalance()       | setBalance()       |

Anyone can read anything and write anything. The same open data, with more typing.

### A surface you designed

|                  |                                     |              |
|------------------|-------------------------------------|--------------|
| getBalance()     | read it                             | setBalance() |
| deposit(amount)  | the only way in                     |              |
| withdraw(amount) | the only way out, and it can refuse |              |

Every setter you add is a rule you decided not to enforce.



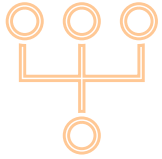
## Three ways to be refused, and two kinds of no

- Every refused construction leaves no object behind
- `withdraw(900)` returns `false`.  
`withdraw(-50)` ends the program
- Before running it, say which is which and why

```
new Account("AC-1001", null, 500.00);  
// Owner name is required
```

```
new Account("AC-1002", "Ada", -100.00);  
// Opening balance cannot be  
// negative: -100.0
```

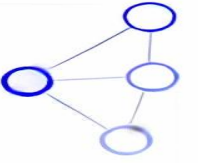
```
ada.withdraw(900.00);    // false  
ada.withdraw(-50.00);   // throws  
// Withdrawal must be positive: -50.0
```



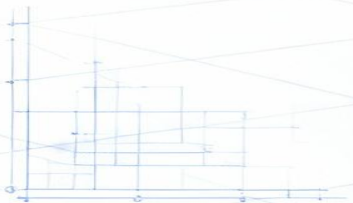
## private is not about **hiding**

- It is about being the only one who can break your own rules
- When the balance is wrong, you read one file instead of forty
- Next: what never changes, and what belongs to the class

```
ada.balance -= 5000.00;  
// balance has private access  
// in Account  
  
ada.accountNumber = "AC-9999";  
// cannot assign a value to  
// final variable accountNumber  
  
// two refusals, two reasons,  
// both of them the design working
```



# What belongs to whom





## Some things are **decided once**

- An account number is chosen when the account opens and never again
- `final` means assigned once, and the compiler refuses the rest
- Not a convention and not a comment. The code does not build

```
private final String accountNumber;  
  
// assigned once, in the constructor  
  
ada.accountNumber = "AC-9999";  
// does not compile, twice over  
  
// ownerName stays changeable,  
// because people really do  
// change their names
```



## A **String** cannot be changed

- Everybody gets caught by this once.  
toUpperCase made a new one
- That is why you can hand a String to any method without worrying
- Nothing can do anything to it, so nothing has to be checked

```
String name = "ada lovelace";  
  
name.toUpperCase();  
System.out.println(name);  
// ada lovelace, unchanged  
  
name = name.toUpperCase();  
// keep the new one
```



## Every field final, no setters, nothing that changes

- Hand it to any code you like. Nobody can alter it
- So nobody has to check what happened to it
- A transaction record should be impossible to edit. Module 3

```
public final class Branch {  
  
    private final String code;  
    private final String city;  
  
    public Branch(String code, String city) {  
        this.code = code;  
        this.city = city;  
    }  
  
    public String getCode() { return code; }  
    public String getCity() { return city; }  
}
```

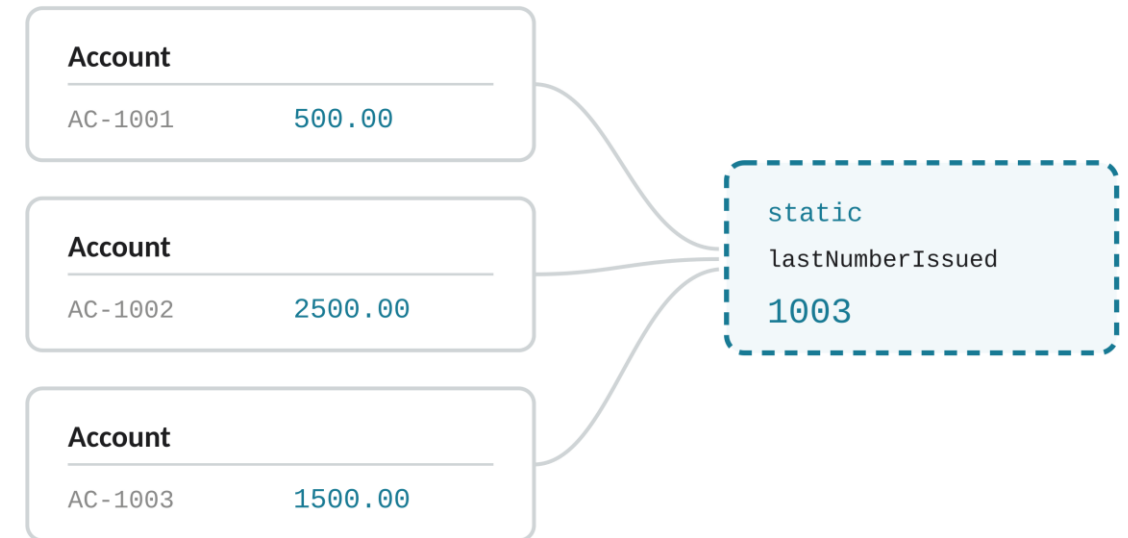


## One counter, however many accounts

- Every account has its own balance, number and owner
- The counter that issues numbers belongs to none of them
- static means exactly one, no matter how many objects exist

One per object

One for the class



Three balances. One counter. Every account reads and updates the same one.



## Validate **before** you touch the counter

- The constructor no longer takes an account number
- Nobody outside can pick one, so nobody can pick one already taken
- Increment first and every refused account burns a number

```
private static int lastNumberIssued =
1000;

public Account(String ownerName,
               double openingBalance) {

    String owner = requireText(
        ownerName, "Owner name");
    // ... the balance check ...

    lastNumberIssued++;
    this.accountNumber =
        "AC-" + lastNumberIssued;
    this.ownerName = owner;
}
```



## A static method **has no object**

- So there is no balance to read. Which account would it be?
- main is static too, which is why you need an object first
- requireText reads no fields, so it should be static as well

```
public static int getAccountsOpened()  
{  
    return lastNumberIssued - 1000;  
}  
  
Account.getAccountsOpened();    // 3  
  
// no this, no instance fields  
  
private static String requireText(  
    String value, String label) {  
    ...  
}
```



## It lives in memory, and memory forgets

- Close the program and lastNumberIssued goes back to 1000

Monday: you run the program

|         |              |
|---------|--------------|
| AC-1001 | Ada Lovelace |
| AC-1002 | Alan Turing  |
| AC-1003 | Grace Hopper |

saved to a file, then the program closes

restart

Tuesday: the counter is back at 1000

AC-1001      Linus Torvalds



that number already belongs to Ada

every lookup now returns whichever it finds first

The counter lives in memory. This is not a bug in the code. It is the honest limit of a counter, and you will meet it in week 11 when you load your accounts back from a file.

- In week 11 you will load accounts from a file and hit this
- Real systems get identifiers from something that survives a restart



## static is not a way to avoid creating objects

- It makes the compiler stop complaining in main, so people spread it
- What you get is one copy of everything and no objects at all
- Finish the sentence: this belongs to the class because \_\_\_\_\_

```
// what people do when main complains
static double balance;
static String ownerName;

static void withdraw(double a) { ... }

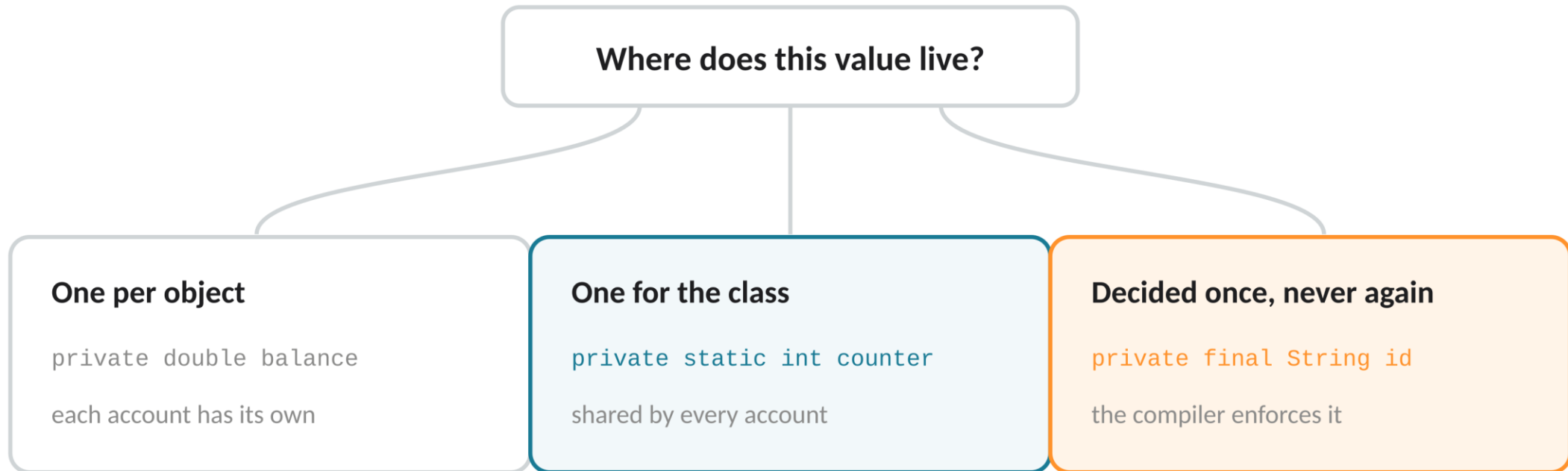
// one copy of everything
// no objects at all

// the week 1 pile of shared data,
// rebuilt on purpose
```



## Where does it live, and when may it change?

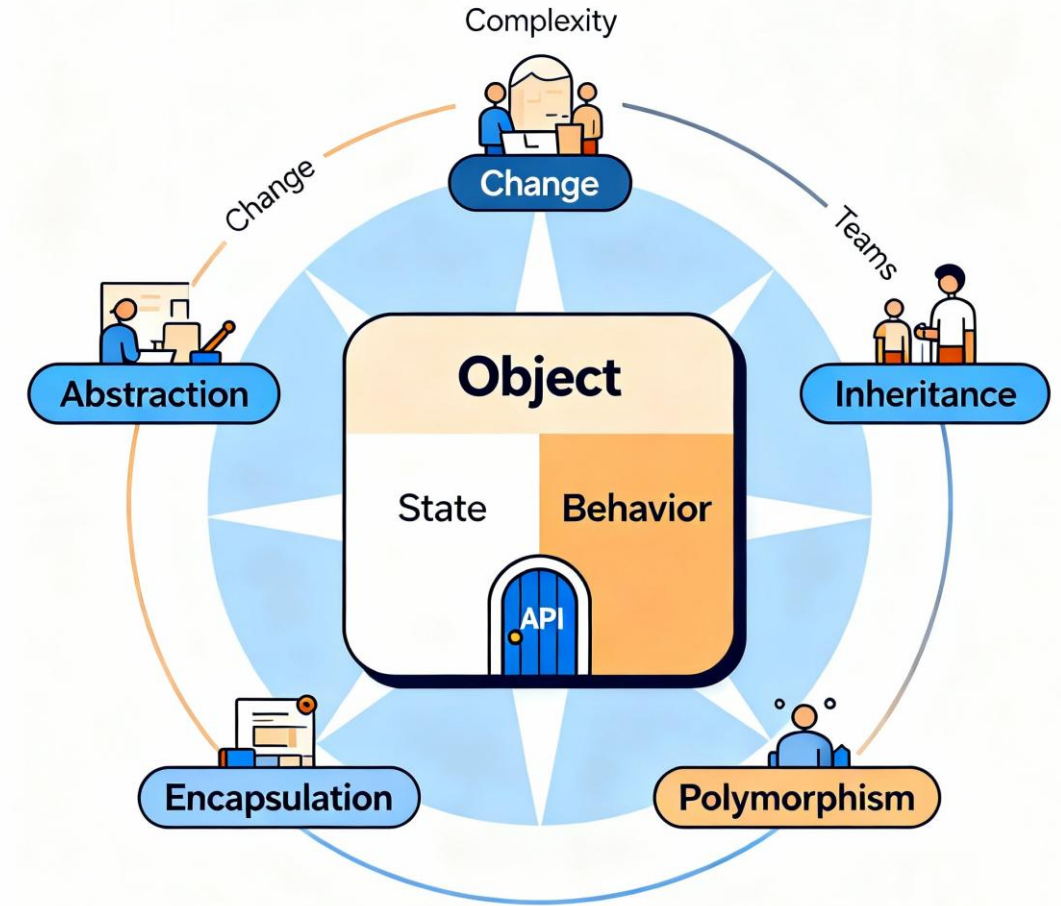
- If you cannot say which of these three it is, you have not decided yet





## Questions with no answer to look up

- A bank merger renumbers every account. Does final make that impossible?
- A String makes a new object on every change. Why accept that cost?
- A customer total could live in three places. Argue for one



# Learn to say no

That's a boundary in life  
and an invariant in OOP.

